PROJECT REQUIREMENTS

Investment Decisions Data Platform

Requirements Gathering Process Guide

PROJECT PURPOSE
The platform collects and organizes AAPL stock-market data, news sentiment, technical indicators, company earnings, and selected macroeconomic indicators through the Alpha Vantage API. The current Python script stores the responses in a local JSON file and periodically records the latest available stock-quote response for later analysis.

Prepared based on the supplied project description and Python script

1. Identify Stakeholders

Stakeholders are the people or parties who use, develop, maintain, or are affected by the platform. The following stakeholders are relevant to the project:

- Project team / developers: implement and maintain the Python data-collection pipeline, handle API errors, and manage the JSON output.
- Data analysts: inspect, clean, combine, and analyze the collected stock, sentiment, technical-indicator, earnings, and economic data.
- Investment researchers or decision-makers: use the prepared data as supporting evidence when studying AAPL and market conditions. The current script does not itself produce investment advice.
- Project supervisor / academic evaluator: reviews whether the project scope, methodology, requirements, and deliverables are clear and appropriate.
- Alpha Vantage API provider: external data source that supplies the requested financial and economic information.
- Potential future platform users: people who may use a dashboard or analysis features if these are added in a later project phase.

Influence and interest mapping:

Stakeholder	Influence	Interest
Project team / developers	High	High — implementation and reliability
Project supervisor	High	High — scope and academic quality
Data analysts	Medium	High — usable, consistent data
Investment researchers / users	Medium to high	High — relevant and trustworthy outputs
Alpha Vantage	High influence on data access	Provider interest in API usage and compliance

2. Plan the Approach

The objective is to build a reliable first-stage data pipeline that collects multiple categories of information related to AAPL and stores them in a structured format for later analysis.

High-level scope:

- In scope: API requests, collection of historical stock prices, news sentiment, selected macroeconomic series, technical indicators, earnings data, JSON storage, and periodic quote snapshots.
- Out of scope for the current script: a dashboard, automated buy/sell recommendations, portfolio or brokerage integration, predictive modeling, backtesting, and trade execution.

Approach and elicitation techniques:

- Project discussions or one-on-one interviews with the supervisor and team to confirm the project goal, expected deliverables, and evaluation criteria.
- A collaborative workshop to agree which datasets are necessary and how they will be stored and used.
- A short survey or feedback session with intended analysts or users, if such users are available, to understand which outputs would be useful.
- A walkthrough of the current Python script and its JSON output to identify existing behavior, limitations, and missing requirements.

Resources and dependencies:

- Python and the requests library, plus standard modules for JSON, timing, file handling, and timestamps.
- Internet access and a valid Alpha Vantage API key.
- A local file named investment_data.json for storing the collected responses.
- Enough time to test endpoint responses, API limits, and the quality of the returned data.

3. Conduct Elicitation

The requirements should be discussed with the project team and supervisor before expanding the implementation. The central business need is to provide one organized source of relevant financial and economic inputs for investment research, rather than simply making isolated API calls.

Questions to ask stakeholders:

- Is AAPL the only company to be supported, or should the platform eventually support multiple ticker symbols?
- Which data categories are essential for the first version: prices, sentiment, macroeconomic indicators, technical indicators, or earnings?
- How frequently must new quote data be collected, and what historical coverage is required?
- Will the JSON file be sufficient, or is a database or dashboard expected?
- What should happen when an endpoint returns an error, rate-limit message, missing fields, or no data?
- What final deliverable is expected: a data pipeline only, a cleaned dataset, a dashboard, or an evaluated analysis method?

KEY REQUIREMENT

The system should gather and preserve the requested data in a form that can later be validated, aligned by date, and analyzed. A collected indicator should not automatically be treated as proof of a profitable investment decision.

4. Document Observations and Findings

The requirements and decisions should be recorded in a shared project document or tracker. The following categories describe the current implementation and important quality needs.

A. Functional requirements — what the system must do

ID	Functional requirement
FR-01	Retrieve daily historical AAPL stock prices through TIME_SERIES_DAILY.
FR-02	Retrieve AAPL-related news sentiment through NEWS_SENTIMENT.
FR-03	Retrieve real GDP, CPI, federal funds rate, 10-year Treasury yield, and unemployment data.
FR-04	Retrieve RSI (14-period), MACD, SMA (20-period), EMA (20-period), and Bollinger Bands for AAPL.
FR-05	Retrieve AAPL earnings data through EARNINGS.
FR-06	Save endpoint responses or clearly marked errors in investment_data.json.
FR-07	Load existing JSON data rather than repeating the initial bulk download when the file already exists.
FR-08	Periodically request GLOBAL_QUOTE and append valid quote snapshots with a retrieval timestamp.
FR-09	Display progress and error messages when API requests or JSON parsing fail.

B. Non-functional requirements — how well the system must work

- Reliability: handle connection failures, HTTP errors, invalid JSON, API error messages, and rate limits clearly.
- Data integrity: preserve existing records and distinguish successful responses from failed or incomplete responses.
- Maintainability: keep ticker, endpoint parameters, and output-file settings easy to update.
- Security: keep the API key outside shared source code and public repositories; use an environment variable or protected local configuration.
- Traceability: preserve timestamps for quote snapshots and, where useful, record retrieval times for other datasets.
- Usability: provide understandable console messages and predictable JSON structure.

C. Project constraints

- The pipeline depends on Alpha Vantage availability, response formats, API plan permissions, and request limits.
- The current implementation is configured for AAPL and stores data locally in JSON.
- The quote-monitoring loop continues until the process is stopped.
- Different datasets have different reporting frequencies, so they cannot be compared safely without appropriate date alignment.

5. Analyze and Prioritize

Requirements should be ranked according to business value, technical feasibility, and dependencies. A reasonable first-version priority is to make collection and storage reliable before adding advanced analysis.

Priority	Requirements	Reason
Must have	Secure API-key handling; fetch configured datasets; validate API responses; save readable JSON; handle errors.	The pipeline needs reliable and safe data collection.
Should have	Consistent timestamps, data-quality checks, explicit handling of rate limits, and clear logging.	These improve trustworthiness and maintainability.
Could have	Dashboard, charts, support for more ticker symbols, database storage, or scheduled deployment.	Useful extensions after the basic pipeline is stable.
Won't have in the first version	Automatic trading, brokerage integration, or unsupported claims of guaranteed investment returns.	These are outside the scope of the supplied script.

The MoSCoW model above is a proposed prioritization for this project. The final priority list should be confirmed with the supervisor and team. Technical dependencies should also be respected: validation and cleaning must happen before dependable combined analysis.

6. Verify and Validate

The documented requirements should be reviewed with the project supervisor and team to ensure that they reflect the expected deliverable. A walkthrough of the code and a sample investment_data.json file can be used to demonstrate current behavior.

- Confirm that each configured API function and parameter matches the intended data requirement.
- Check whether successful responses contain the expected endpoint-specific data fields.
- Test what happens when the API returns a rate-limit message, an error message, an empty response, or a network failure.
- Verify that saved JSON is valid and existing quote snapshots are preserved when new ones are appended.
- Confirm the expected refresh frequency, historical data coverage, and project deliverables with the supervisor.
- Record approval of the requirements and any agreed exclusions before implementing major new features.

SUGGESTED SIGN-OFF
The project supervisor and project team should approve the scope, priority list, expected output, and acceptance criteria. This is a proposed step; approval has not been assumed to have occurred.

7. Manage Changes

The team should maintain an approved baseline of the requirements and a traceability table that links each requirement to its implementation and test. If a new feature is requested—for example, adding another stock symbol or a dashboard—the team should record the request, assess its value and impact, and approve or defer it before changing the scope.

Requirement ID	Implementation item	Verification / traceability check
FR-01	Historical stock prices collected	Check that the response includes the expected daily time-series field.
FR-06	JSON storage	Confirm the file parses correctly and failed

		responses are clearly marked.
FR-08	Latest quote snapshots	Confirm a valid quote is appended with retrieved_at and previous snapshots remain.
NFR-01	Reliability and rate limits	Test API errors, timeouts, invalid JSON, and provider rate-limit responses.
NFR-02	API-key security	Verify the key is not stored in source code committed to a shared repository.

Proposed change-control steps:

- Record the requested change and the reason for it.
- Assess the effect on scope, schedule, data requirements, security, and testing.
- Ask the responsible team or supervisor to approve, reject, or defer the change.
- Update the requirements baseline and traceability table after approval.
- Implement and test the change, then document the result.

Final Summary

For the Investment Decisions Data Platform, requirements gathering establishes who needs the platform, what financial and economic data it should collect, how that data should be stored and checked, which features should be implemented first, and how the team will verify and manage the requirements. The supplied Python code provides the starting point: an AAPL-focused Alpha Vantage data collector with JSON storage and periodic quote snapshots. The next priority is to confirm scope with stakeholders and improve validation, reliability, and secure handling before building advanced analysis or decision-support features.